

Symbolic Attitude Dynamics for Spacecraft Slew Maneuver Planning

Derive closed-form slew time equations, rate profiles, and pointing sensitivity from Euler's rotational dynamics. Using closed-form parametric analysis answers the "how long does it take to turn?" question without running a simulation for every scenario.

Workflow: Euler's equations from first principles → eigenaxis reduction to scalar ODE → `dsolve` for exact bang-bang and rate-limited slew profiles → closed-form slew time as a symbolic function of inertia, torque, rate limit, and angle → `matlabFunction` code generation → sensitivity analysis for inertia growth and actuator degradation → pointing budget with `vpa` → mission timeline integration

Requires: Symbolic Math Toolbox™, Aerospace Toolbox (for quaternion)

Copyright 2026 The MathWorks, Inc.

1. Euler's Equations of Rotational Motion

For a rigid spacecraft with body-fixed principal axes, the rotational dynamics are governed by Euler's equations. The angular momentum $\vec{H} = \mathbf{I}\vec{\omega}$ evolves under applied torque $\vec{\tau}$:

$$\dot{\vec{H}} + \vec{\omega} \times (\mathbf{I}\vec{\omega}) = \vec{\tau}$$

where $\mathbf{I} = \text{diag}(I_x, I_y, I_z)$ is the principal moment of inertia tensor. Expanding in component form:

$$I_x \dot{\omega}_x + (I_z - I_y)\omega_y\omega_z = \tau_x$$

$$I_y \dot{\omega}_y + (I_x - I_z)\omega_z\omega_x = \tau_y$$

$$I_z \dot{\omega}_z + (I_y - I_x)\omega_x\omega_y = \tau_z$$

The cross-coupling terms $(I_z - I_y)\omega_y\omega_z$ etc. are the gyroscopic torques that make 3D rotational dynamics nonlinear. For single-axis slews, these terms vanish.

```
syms Ix Iy Iz positive
syms wx wy wz real
syms taux tauy tauz real
I_tensor = diag([Ix, Iy, Iz]);
```

```
omega_vec = [wx; wy; wz];
tau_vec = [taux; tauy; tauz];
euler_eqs = I_tensor * sym('alpha', [3 1]) + cross(omega_vec, I_tensor * omega_vec) - tau_vec;
```

Euler's equations (principal axes, $\tau = I\alpha + \omega \times H$):

```
euler_eqs_explicit = [
    Ix * sym('alpha_x') + (Iz - Iy)*wy*wz -iaux;
    Iy * sym('alpha_y') + (Ix - Iz)*wz*wx -iauy;
    Iz * sym('alpha_z') + (Iy - Ix)*wx*wy -iauz]
```

```
euler_eqs_explicit =

$$\begin{bmatrix} I_x \alpha_x - \tau_{ax} - w_y w_z (I_y - I_z) \\ I_y \alpha_y - \tau_{ay} + w_x w_z (I_x - I_z) \\ I_z \alpha_z - \tau_{az} - w_x w_y (I_x - I_y) \end{bmatrix}$$

```

2. Eigenaxis Slew - Reduction to Scalar Dynamics

For a rotation purely about one principal axis (the "eigenaxis"), the cross-coupling terms vanish because the other two angular velocity components are zero. This is the standard maneuver mode for reaction-wheel-based ADCS: the flight software computes the eigenaxis (shortest-path rotation axis) and commands torque along it.

With rotation about a single axis with moment of inertia I , the dynamics reduce to:

$$I\ddot{\theta} = \tau(t)$$

This is a simple double-integrator: torque produces angular acceleration, which integrates to rate, which integrates to angle. The slew planning problem becomes: given constraints on maximum torque $\tau_{\max}$ and maximum angular rate $\omega_{\max}$, what is the minimum time to rotate through angle Θ ?

```
syms I_ax positive
syms t
syms theta(t)
Dtheta = diff(theta, t);
D2theta = diff(theta, t, 2);
syms tau_applied
```

Eigenaxis equation of motion:

```
eigenaxis_eom = I_ax * D2theta == tau_applied
```

```
eigenaxis_eom(t) =
```

$$I_{\text{ax}} \frac{\partial^2}{\partial t^2} \theta(t) = \tau_{\text{applied}}$$

3. Time-Optimal (Bang-Bang) Slew Profile

When the slew is limited only by available torque $\tau_{\max}$ (no rate constraint), the time-optimal solution is a **bang-bang** profile: full positive torque for the first half of the slew, then full negative torque to decelerate to rest. The angular acceleration is $\pm\alpha_{\max} = \tau_{\max}/I$.

Phase 1 (accelerate, $0 \leq t \leq t_s/2$): $\ddot{\theta} = +\alpha_{\max}$

Phase 2 (decelerate, $t_s/2 \leq t \leq t_s$): $\ddot{\theta} = -\alpha_{\max}$

Solve each phase with `dsolve` to get the exact angle and rate profiles.

```
syms alpha_max positive
syms Theta positive
syms ts positive
syms t_var
syms th1(t_var)
Dth1 = diff(th1, t_var);
sol_phase1 = dsolve(diff(th1, t_var, 2) == alpha_max, ...
    th1(0) == 0, Dth1(0) == 0);
```

Phase 1 (accelerate) - angle vs time:

```
theta_p1 = simplify(sol_phase1)
```

```
theta_p1 =
```

$$\frac{\alpha_{\max} t_{\text{var}}^2}{2}$$

Phase 1 - rate vs time:

```
omega_p1 = simplify(diff(sol_phase1, t_var))
```

$$\omega_{p1} = \alpha_{\max} t_{\text{var}}$$

Closed-Form Bang-Bang Slew Time

At the midpoint $t = t_s/2$, the spacecraft has rotated half the total angle $\Theta/2$ and reached peak rate ω_{peak} . Solving $\Theta/2 = \frac{1}{2} \alpha_{\max} (t_s/2)^2$ for t_s :

```
syms t_slew_bb positive
half_angle_eq = alpha_max/2 * (t_slew_bb/2)^2 == Theta/2;
```

Bang-bang slew time (torque-limited):

```
t_slew_bb_sol = solve(half_angle_eq, t_slew_bb)
```

t_slew_bb_sol =

$$\frac{2 \sqrt{\Theta}}{\sqrt{\alpha_{\max}}}$$

Peak angular rate during bang-bang slew:

```
omega_peak_bb = simplify(alpha_max * t_slew_bb_sol / 2)
```

$$\omega_{\text{peak_bb}} = \sqrt{\Theta} \sqrt{\alpha_{\max}}$$

Express in Terms of Physical Parameters

Substituting $\alpha_{\max} = \tau_{\max}/I$ gives slew time and peak rate as functions of the spacecraft's physical properties:

```
syms tau_max positive
```

Slew time $t_s = f(\Theta, I, \tau_{\max})$:

```
t_bb_physical = simplify(subs(t_slew_bb_sol, alpha_max, tau_max/I_ax))
```

```
t_bb_physical =
```

$$\frac{2 \sqrt{I_{ax}} \sqrt{\Theta}}{\sqrt{\tau_{max}}}$$

Peak rate omega_peak = f(Theta, I, tau_max):

```
omega_peak_physical = simplify(subs(omega_peak_bb, alpha_max, tau_max/I_ax))
```

```
omega_peak_physical =
```

$$\frac{\sqrt{\Theta} \sqrt{\tau_{max}}}{\sqrt{I_{ax}}}$$

4. Rate-Limited (Trapezoidal) Slew Profile

Spacecraft have a maximum angular rate ω_{max} imposed by star tracker acquisition limits, structural flex excitation, or reaction wheel momentum saturation. When the bang-bang peak rate exceeds ω_{max} , the profile becomes **trapezoidal**: accelerate at α_{max} until hitting the rate limit, coast at ω_{max} , then decelerate.

Phase 1 (accelerate): $0 \rightarrow \omega_{max}$ in time $t_a = \omega_{max}/\alpha_{max}$

Phase 2 (coast): hold ω_{max} for time t_c

Phase 3 (decelerate): $\omega_{max} \rightarrow 0$ in time t_a (symmetric)

The total angle constraint $\Theta = \alpha_{max}t_a^2 + \omega_{max}t_c$ determines the coast time.

```
syms omega_max positive
syms t_accel t_coast
t_accel_expr = omega_max / alpha_max;
theta_accel = alpha_max / 2 * t_accel_expr^2;
theta_coast_eq = Theta == 2*theta_accel + omega_max * t_coast;
t_coast_sol = solve(theta_coast_eq, t_coast);
```

Acceleration time t_a :

```
t_accel_expr
```

$t_{\text{accel_expr}} =$

$$\frac{\omega_{\max}}{\alpha_{\max}}$$

Coast time t_c :

```
simplify(t_coast_sol)
```

$\text{ans} =$

$$\frac{\Theta - \frac{\omega_{\max}^2}{\alpha_{\max}}}{\omega_{\max}}$$

```
t_slew_trap = simplify(2*t_accel_expr + t_coast_sol);
```

Total trapezoidal slew time:

```
t_slew_trap
```

$t_{\text{slew_trap}} =$

$$\frac{\omega_{\max}^2 + \Theta \alpha_{\max}}{\alpha_{\max} \omega_{\max}}$$

Unified Slew Time - Regime Selection

The trapezoidal profile applies only when $\omega_{\text{peak,bb}} > \omega_{\max}$. Otherwise the bang-bang profile completes before the rate limit is reached. The crossover angle Θ^* where the regimes switch is:

$$\Theta^* = \omega_{\max}^2 / \alpha_{\max}$$

Below Θ^* the slew is torque-limited (bang-bang); above it, rate-limited (trapezoidal).

Crossover angle (bang-bang ↔ trapezoidal):

```
Theta_crossover = simplify(omega_max^2 / alpha_max)
```

```
Theta_crossover =
```

$$\frac{\omega_{\max}^2}{\alpha_{\max}}$$

```
t_slew_unified = piecewise(Theta <= Theta_crossover, ...
    subs(t_slew_bb_sol, alpha_max, alpha_max), ...
    Theta > Theta_crossover, t_slew_trap);
```

Unified slew time expression:

```
t_slew_unified = simplify(t_slew_unified)
```

```
t_slew_unified =
```

$$\begin{cases} \frac{2\sqrt{\Theta}}{\sqrt{\alpha_{\max}}} & \text{if } \Theta \alpha_{\max} \leq \omega_{\max}^2 \\ \frac{\omega_{\max}^2 + \Theta \alpha_{\max}}{\alpha_{\max} \omega_{\max}} & \text{if } \omega_{\max}^2 < \Theta \alpha_{\max} \end{cases}$$

5. Parametric Slew Time

The unified slew time is now a closed-form symbolic function of four parameters: slew angle Θ , moment of inertia I , maximum torque $\tau_{\max}$, and maximum rate $\omega_{\max}$. This parametric form lets mission planners explore the entire design space without re-running anything.

```
t_slew_full = subs(t_slew_unified, alpha_max, tau_max/I_ax);
```

Slew time $t_s(\Theta, I, \tau_{\max}, \omega_{\max})$:

```
t_slew_full = simplify(t_slew_full)
```

```
t_slew_full =
```

$$\left\{ \begin{array}{ll} \frac{2 \sqrt{I_{ax}} \sqrt{\Theta}}{\sqrt{\tau_{max}}} & \text{if } \Theta \tau_{max} \leq I_{ax} \omega_{max}^2 \\ \frac{I_{ax} \omega_{max}^2 + \Theta \tau_{max}}{\omega_{max} \tau_{max}} & \text{if } I_{ax} \omega_{max}^2 < \Theta \tau_{max} \end{array} \right.$$

```
Theta_cross_phys = simplify(subs(Theta_crossover, alpha_max, tau_max/I_ax));
```

Regime crossover angle (physical parameters):

Theta_cross_phys

Theta_cross_phys =

$$\frac{I_{ax} \omega_{max}^2}{\tau_{max}}$$

6. Reference Mission: Agile Earth-Observing Satellite

Evaluate for a high-resolution agile imaging satellite performing rapid cross-track retargeting. This class of spacecraft (e.g., Pleiades Neo, WorldView Legion) requires fast slews to maximize daily imaging capacity.

Parameter	Value	Rationale
Bus mass	500 kg	Typical agile EO platform
Pitch inertia	120 kg·m ²	Pitch axis (cross-track slew)
Reaction wheel torque	0.3 N·m	4-wheel pyramid, single-axis component
Max angular rate	3.0 deg/s	Star tracker tracking limit
Slew angle	30 deg (nominal), up to 45 deg (off-nadir)	

```
I_val = 120;
tau_max_val = 0.3;
omega_max_val = deg2rad(3.0);
Theta_nom = deg2rad(30);
alpha_max_val = tau_max_val / I_val;
Theta_cross_val = omega_max_val^2 / alpha_max_val;
```

```

fprintf(['=== Reference Mission: Agile EO Satellite ===\n' ...
        'Pitch inertia:      %.0f kg·m²\n' ...
        'Available torque:   %.2f N·m\n' ...
        'Max angular accel:  %.4f deg/s²\n' ...
        'Max angular rate:   %.1f deg/s\n' ...
        'Crossover angle:    %.2f deg\n'], ...
        I_val, ...
        tau_max_val, ...
        rad2deg(alpha_max_val), ...
        rad2deg(omega_max_val), ...
        rad2deg(Theta_cross_val))

```

```

=== Reference Mission: Agile EO Satellite ===
Pitch inertia:      120 kg·m²
Available torque:   0.30 N·m
Max angular accel:  0.1432 deg/s²
Max angular rate:   3.0 deg/s
Crossover angle:    62.83 deg

```

```

if Theta_nom > Theta_cross_val
    regime_str = 'RATE-LIMITED';
else
    regime_str = 'TORQUE-LIMITED';
end
fprintf('Nominal slew:      %.0f deg → %s regime\n', rad2deg(Theta_nom), regime_str)

```

```

Nominal slew:      30 deg → TORQUE-LIMITED regime

```

Evaluate Slew Times

Compute exact slew times for typical imaging scenarios using the closed-form expressions.

```

slew_angles_deg = [5, 10, 15, 20, 30, 45];
slew_angles_rad = deg2rad(slew_angles_deg);
t_slew_vals = zeros(size(slew_angles_rad));
for k = 1:length(slew_angles_rad)
    ang = slew_angles_rad(k);
    if ang <= Theta_cross_val
        t_slew_vals(k) = 2*sqrt(ang / alpha_max_val);
    end
end

```

```

else
    t_a = omega_max_val / alpha_max_val;
    t_c = (ang - omega_max_val^2/alpha_max_val) / omega_max_val;
    t_slew_vals(k) = 2*t_a + t_c;
end
end
fprintf(['\n=== Slew Times (closed-form) ===\n' ...
        ' Angle [deg]   Time [s]   Regime\n'])

=== Slew Times (closed-form) ===
Angle [deg]   Time [s]   Regime

for k = 1:length(slew_angles_deg)
    if slew_angles_rad(k) > Theta_cross_val
        regime = 'rate-lim';
    else
        regime = 'torque-lim';
    end
    fprintf(' %5.0f°      %6.2f      %s\n', slew_angles_deg(k), t_slew_vals(k), regime)
end

```

5°	11.82	torque-lim
10°	16.71	torque-lim
15°	20.47	torque-lim
20°	23.63	torque-lim
30°	28.94	torque-lim
45°	35.45	torque-lim

7. Code Generation: Deployable Slew Planner

Convert the symbolic slew time expressions into optimized MATLAB® functions via `matlabFunction`. The piecewise slew time has two regimes (bang-bang and trapezoidal), so generate each branch separately and combine them in a vectorized wrapper that uses logical indexing.

```

t_bb_expr = subs(t_slew_bb_sol, alpha_max, tau_max/I_ax);
t_trap_expr = subs(t_slew_trap, alpha_max, tau_max/I_ax);
Theta_cross_expr = I_ax * omega_max^2 / tau_max;
matlabFunction(t_bb_expr, t_trap_expr, Theta_cross_expr, ...
    'File', 'slewTimeKernels', ...
    'Vars', {Theta, I_ax, tau_max, omega_max}, ...

```

```

    'Outputs', {'t_bb', 't_trap', 'Theta_star'}));
slewTimeCode = [...
    "function t_slew = slewTime(Theta, I, tau_max, omega_max)"
    "% SLEWTIME Closed-form slew time (vectorized over Theta).\"
    \"% t_slew = slewTime(Theta, I, tau_max, omega_max)\"
    \"% Handles both bang-bang and rate-limited regimes.\"
    "[t_bb, t_trap, Theta_star] = slewTimeKernels(Theta, I, tau_max, omega_max);\"
    \"t_slew = t_bb;\"
    \"mask = Theta > Theta_star;\"
    \"t_slew(mask) = t_trap(mask);\"
    \"end\"];
writelines(slewTimeCode, 'slewTime.m');
fprintf(['Generated: slewTimeKernels.m (symbolic core)\n' ...
    'Generated: slewTime.m (vectorized wrapper)\n' ...
    ' Signature: t_slew = slewTime(Theta, I, tau_max, omega_max)\n' ...
    ' Accepts vectorized Theta for batch evaluation\n'])

```

```

Generated: slewTimeKernels.m (symbolic core)
Generated: slewTime.m (vectorized wrapper)
Signature: t_slew = slewTime(Theta, I, tau_max, omega_max)
Accepts vectorized Theta for batch evaluation

```

Generate Slew Profile Functions

For detailed profile visualization and downstream ADCS simulation, generate functions that return the angular rate profile (trapezoidal or triangular) and the angle-vs-time trajectory.

```

syms t_eval positive
omega_profile_trap = piecewise(...
    t_eval <= omega_max/alpha_max, alpha_max * t_eval, ...
    t_eval <= omega_max/alpha_max + (Theta - omega_max^2/alpha_max)/omega_max, omega_max, ...
    symtrue, omega_max - alpha_max * (t_eval - omega_max/alpha_max - (Theta - omega_max^2/alpha_max)/omega_max));
omega_profile_bb = piecewise(...
    t_eval <= sqrt(Theta/alpha_max), alpha_max * t_eval, ...
    symtrue, alpha_max * (2*sqrt(Theta/alpha_max) - t_eval));
matlabFunction(omega_profile_trap, ...
    'File', 'slewRateProfileTrap', ...
    'Vars', {t_eval, Theta, alpha_max, omega_max}, ...

```

```

    'Outputs', {'omega'}));
matlabFunction(omega_profile_bb, ...
    'File', 'slewRateProfileBB', ...
    'Vars', {t_eval, alpha_max, Theta}, ...
    'Outputs', {'omega'}));
fprintf(['Generated: slewRateProfileTrap.m\n' ...
    'Generated: slewRateProfileBB.m\n'])

```

Generated: slewRateProfileTrap.m
Generated: slewRateProfileBB.m

Visualize the Slew Profile for the 30° Reference Maneuver

The profile shape depends on whether the maneuver is torque-limited (bang-bang/triangular) or rate-limited (trapezoidal). At 30° with a crossover of ~63°, this maneuver is torque-limited: the spacecraft never reaches the rate limit.

```

if Theta_nom < Theta_cross_val
    % Bang-bang (triangular) regime
    t_half_val = sqrt(Theta_nom / alpha_max_val);
    t_total_val = 2 * t_half_val;
    omega_peak_val = alpha_max_val * t_half_val;
    t_profile = linspace(0, t_total_val, 500);
    omega_profile = zeros(size(t_profile));
    theta_profile = zeros(size(t_profile));
    for k = 1:length(t_profile)
        tk = t_profile(k);
        if tk <= t_half_val
            omega_profile(k) = alpha_max_val * tk;
            theta_profile(k) = 0.5 * alpha_max_val * tk^2;
        else
            dt = tk - t_half_val;
            omega_profile(k) = omega_peak_val - alpha_max_val * dt;
            theta_profile(k) = 0.5*alpha_max_val*t_half_val^2 + ...
                omega_peak_val*dt - 0.5*alpha_max_val*dt^2;
        end
    end
    profile_label = 'Bang-Bang (triangular)';
else

```

```

% Trapezoidal regime
t_a_val = omega_max_val / alpha_max_val;
t_c_val = (Theta_nom - omega_max_val^2/alpha_max_val) / omega_max_val;
t_total_val = 2*t_a_val + t_c_val;
omega_peak_val = omega_max_val;
t_profile = linspace(0, t_total_val, 500);
omega_profile = zeros(size(t_profile));
theta_profile = zeros(size(t_profile));
for k = 1:length(t_profile)
    tk = t_profile(k);
    if tk <= t_a_val
        omega_profile(k) = alpha_max_val * tk;
        theta_profile(k) = 0.5 * alpha_max_val * tk^2;
    elseif tk <= t_a_val + t_c_val
        omega_profile(k) = omega_max_val;
        theta_profile(k) = 0.5*alpha_max_val*t_a_val^2 + omega_max_val*(tk - t_a_val);
    else
        dt = tk - t_a_val - t_c_val;
        omega_profile(k) = omega_max_val - alpha_max_val * dt;
        theta_profile(k) = 0.5*alpha_max_val*t_a_val^2 + omega_max_val*t_c_val + ...
            omega_max_val*dt - 0.5*alpha_max_val*dt^2;
    end
end
profile_label = 'Trapezoidal (rate-limited)';
end
fprintf(['Profile regime:      %s\n' ...
        'Peak angular rate:   %.2f deg/s (limit: %.1f deg/s)\n' ...
        'Slew time:           %.2f s\n'], ...
        profile_label, rad2deg(omega_peak_val), rad2deg(omega_max_val), t_total_val)

```

```

Profile regime:      Bang-Bang (triangular)
Peak angular rate:   2.07 deg/s (limit: 3.0 deg/s)
Slew time:           28.94 s

```

```

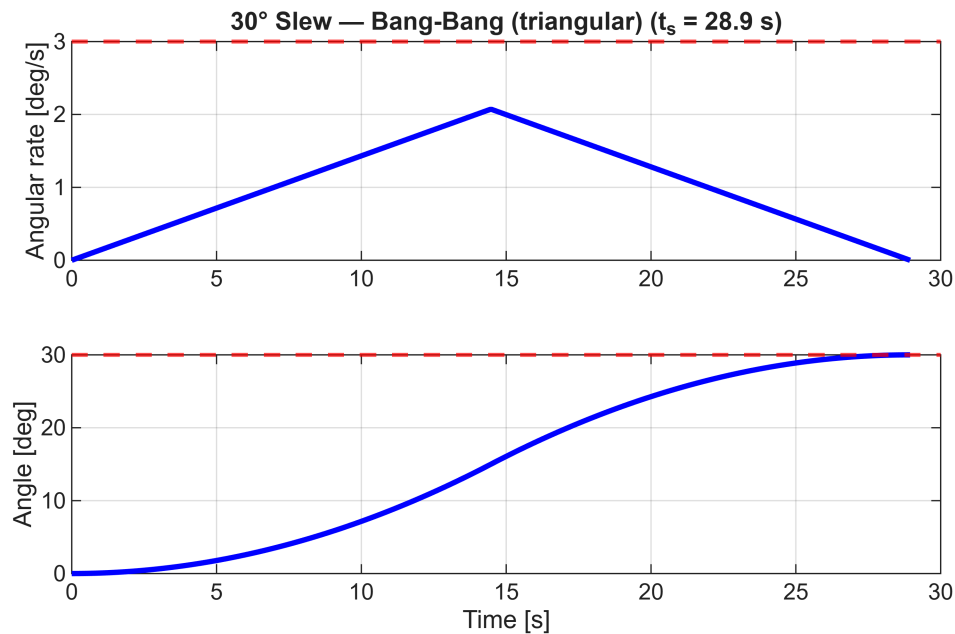
tiledlayout(2,1)
nexttile
plot(t_profile, rad2deg(omega_profile), 'b-', 'LineWidth', 2)
hold on

```

```

yline(rad2deg(omega_max_val), 'r--', '\omega_{max} (limit)', 'LineWidth', 1.5)
hold off
ylabel('Angular rate [deg/s]')
title(sprintf('30° Slew — %s (t_s = %.1f s)', profile_label, t_total_val))
grid on
nexttile
plot(t_profile, rad2deg(theta_profile), 'b-', 'LineWidth', 2)
hold on
yline(30, 'r--', '30° target', 'LineWidth', 1.5)
hold off
xlabel('Time [s]')
ylabel('Angle [deg]')
grid on

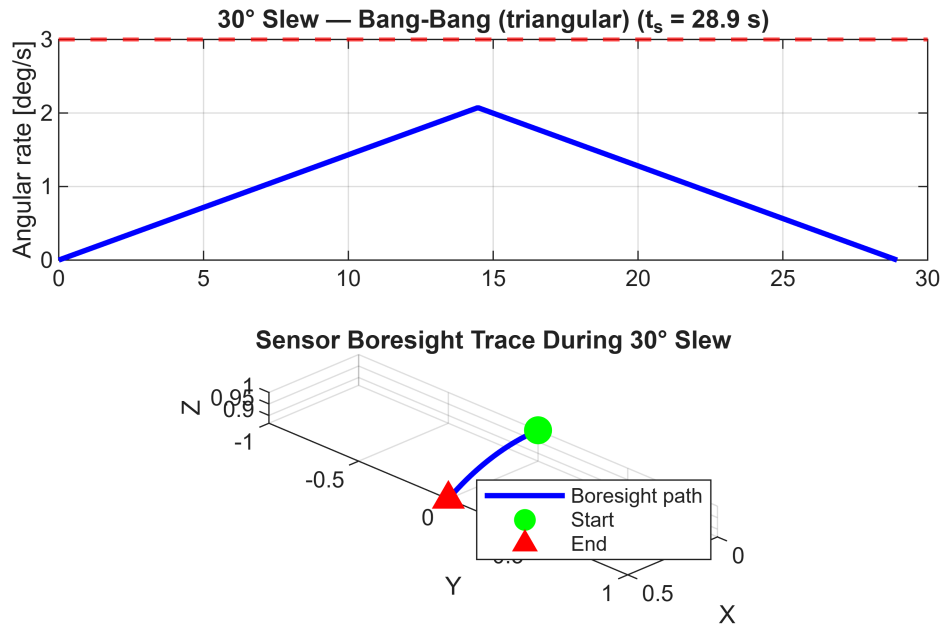
```



Aerospace Toolbox Integration: Quaternion Attitude Propagation

The symbolically-derived angle profile feeds directly into Aerospace Toolbox quaternion objects for 3D attitude representation. This bridges the gap between the 1D eigenaxis planning (symbolic) and full 3D attitude simulation (numeric). The eigenaxis for a cross-track slew is the pitch axis (body Y).

```
slew_axis = [0 1 0]; % pitch axis (body-Y) for cross-track imaging slew
n_pts = 50;
t_quat = linspace(0, t_total_val, n_pts);
theta_quat = interp1(t_profile, theta_profile, t_quat);
rotvecs = theta_quat(:) * slew_axis; % N×3 rotation vectors (angle * axis)
q_slew = quaternion(rotvecs, 'rotvec');
% Propagate a body-fixed vector (e.g., sensor boresight along body-Z)
boresight_body = [0 0 1];
boresight_inertial = rotatepoint(q_slew, boresight_body);
plot3(boresight_inertial(:,1), boresight_inertial(:,2), boresight_inertial(:,3), ...
      'b-', 'LineWidth', 2)
hold on
plot3(boresight_inertial(1,1), boresight_inertial(1,2), boresight_inertial(1,3), ...
      'go', 'MarkerSize', 10, 'MarkerFaceColor', 'g')
plot3(boresight_inertial(end,1), boresight_inertial(end,2), boresight_inertial(end,3), ...
      'r^', 'MarkerSize', 10, 'MarkerFaceColor', 'r')
hold off
xlabel('X'); ylabel('Y'); zlabel('Z')
title('Sensor Boresight Trace During 30° Slew')
legend('Boresight path', 'Start', 'End', 'Location', 'best')
grid on
axis equal
view(135, 25)
```



8. Sensitivity Analysis: Inertia Growth & Actuator Degradation

Over mission life, spacecraft inertia changes (fuel depletion, solar array deployment, appendage thermal distortion) and reaction wheel torque degrades (bearing friction, driver aging). The symbolic slew time gives exact partial derivatives — no finite-difference approximation.

Inertia Sensitivity

How does slew time change per unit increase in inertia?

```
dt_dI = diff(t_slew_full, I_ax);
```

$\partial t_{\text{slew}} / \partial I$ (sensitivity to inertia):

```
dt_dI = simplify(dt_dI)
```

```
dt_dI =
```

$$\begin{cases} \frac{\sqrt{\Theta}}{\sqrt{I_{ax}} \sqrt{\tau_{max}}} & \text{if } \Theta \tau_{max} < I_{ax} \omega_{max}^2 \\ \frac{\omega_{max}}{\tau_{max}} & \text{if } I_{ax} \omega_{max}^2 < \Theta \tau_{max} \end{cases}$$

Torque Sensitivity

How does slew time change if available torque decreases (wheel degradation)?

```
dt_dtau = diff(t_slew_full, tau_max);
```

$\partial t_{\text{slew}} / \partial \tau_{\text{max}}$ (sensitivity to available torque):

```
dt_dtau = simplify(dt_dtau)
```

dt_dtau =

$$\begin{cases} -\frac{\sqrt{I_{ax}} \sqrt{\Theta}}{\tau_{max}^{3/2}} & \text{if } \Theta \tau_{max} < I_{ax} \omega_{max}^2 \\ -\frac{I_{ax} \omega_{max}}{\tau_{max}^2} & \text{if } I_{ax} \omega_{max}^2 < \Theta \tau_{max} \end{cases}$$

Rate Limit Sensitivity

```
dt_domega = diff(t_slew_full, omega_max);
```

$\partial t_{\text{slew}} / \partial \omega_{\text{max}}$ (sensitivity to rate limit):

```
dt_domega = simplify(dt_domega)
```

dt_domega =

$$\begin{cases} 0 & \text{if } \Theta \tau_{max} < I_{ax} \omega_{max}^2 \\ -\frac{\Theta \tau_{max} - I_{ax} \omega_{max}^2}{\omega_{max}^2 \tau_{max}} & \text{if } I_{ax} \omega_{max}^2 < \Theta \tau_{max} \end{cases}$$

Generate Deployable Sensitivity Functions

```
matlabFunction(dt_dI, dt_dtau, dt_domega, ...
    'File', 'slewTimeSensitivity', ...
    'Vars', {Theta, I_ax, tau_max, omega_max}, ...
    'Outputs', {'dts_dI', 'dts_dtau', 'dts_domega'}));
fprintf('Generated: slewTimeSensitivity.m\n')
```

Generated: slewTimeSensitivity.m

Evaluate Sensitivity at Reference Conditions

Quantify the impact of realistic degradation scenarios: +10% inertia growth (fuel redistribution) and -15% torque loss (wheel aging at year 5).

```
[dts_dI_val, dts_dtau_val, dts_domega_val] = slewTimeSensitivity(...
    Theta_nom, I_val, tau_max_val, omega_max_val);
fprintf(['\n=== Sensitivity at 30° Slew ===\n' ...
    '\n∂t/∂I:          %+.4f s per kg·m²\n' ...
    '\n∂t/∂tau_max:    %+.4f s per N·m\n' ...
    '\n∂t/∂omega_max: %+.4f s per rad/s\n'], ...
    dts_dI_val, ...
    dts_dtau_val, ...
    dts_domega_val)
```

```
=== Sensitivity at 30° Slew ===
∂t/∂I:          +0.1206 s per kg·m²
∂t/∂tau_max:    -48.2401 s per N·m
∂t/∂omega_max: +0.0000 s per rad/s
```

```
delta_I = 0.10 * I_val;
delta_tau = -0.15 * tau_max_val;
dt_inertia_growth = dts_dI_val * delta_I;
dt_torque_loss = dts_dtau_val * delta_tau;
fprintf(['\n=== Mission-Life Degradation Impact (30° slew) ===\n' ...
    '+10%% inertia:    %+.2f s  (%.1f%% increase)\n' ...
    '-15%% torque:     %+.2f s  (%.1f%% increase)\n' ...
    'Combined:        %+.2f s  (%.1f%% increase)\n'], ...
    dt_inertia_growth, 100*dt_inertia_growth/t_total_val, ...
```

```
dt_torque_loss, 100*dt_torque_loss/t_total_val, ...
dt_inertia_growth + dt_torque_loss, ...
100*(dt_inertia_growth + dt_torque_loss)/t_total_val)
```

```
=== Mission-Life Degradation Impact (30° slew) ===
+10% inertia:  +1.45 s  (5.0% increase)
-15% torque:   +2.17 s  (7.5% increase)
Combined:      +3.62 s  (12.5% increase)
```

Validation via Variable-Precision Arithmetic

The symbolic derivative is exact by construction, but we can verify it independently: evaluate the slew time expression at I and $I + \delta$ using 32-digit arithmetic, then compare the finite-difference quotient against the analytic sensitivity. This is a critical step in certification workflows.

```
delta_sym = sym(1)/1000; % small perturbation (exact rational)
I_sym = sym(I_val);
tau_sym = sym(tau_max_val);
omega_sym = sym(omega_max_val);
Theta_sym = sym(Theta_nom);
t_at_I = vpa(subs(t_slew_full, [Theta, I_ax, tau_max, omega_max], ...
    [Theta_sym, I_sym, tau_sym, omega_sym]), 32);
t_at_I_plus = vpa(subs(t_slew_full, [Theta, I_ax, tau_max, omega_max], ...
    [Theta_sym, I_sym + delta_sym, tau_sym, omega_sym]), 32);
fd_sensitivity = (t_at_I_plus - t_at_I) / delta_sym;
analytic_sensitivity = vpa(subs(dt_dI, [Theta, I_ax, tau_max, omega_max], ...
    [Theta_sym, I_sym, tau_sym, omega_sym]), 32);
fprintf(['\n=== VPA Verification (32-digit) ===\n' ...
    'Analytic ∂t/∂I:      %s\n' ...
    'Finite-diff (δI=0.001): %s\n'], ...
    char(analytic_sensitivity), ...
    char(fd_sensitivity))
```

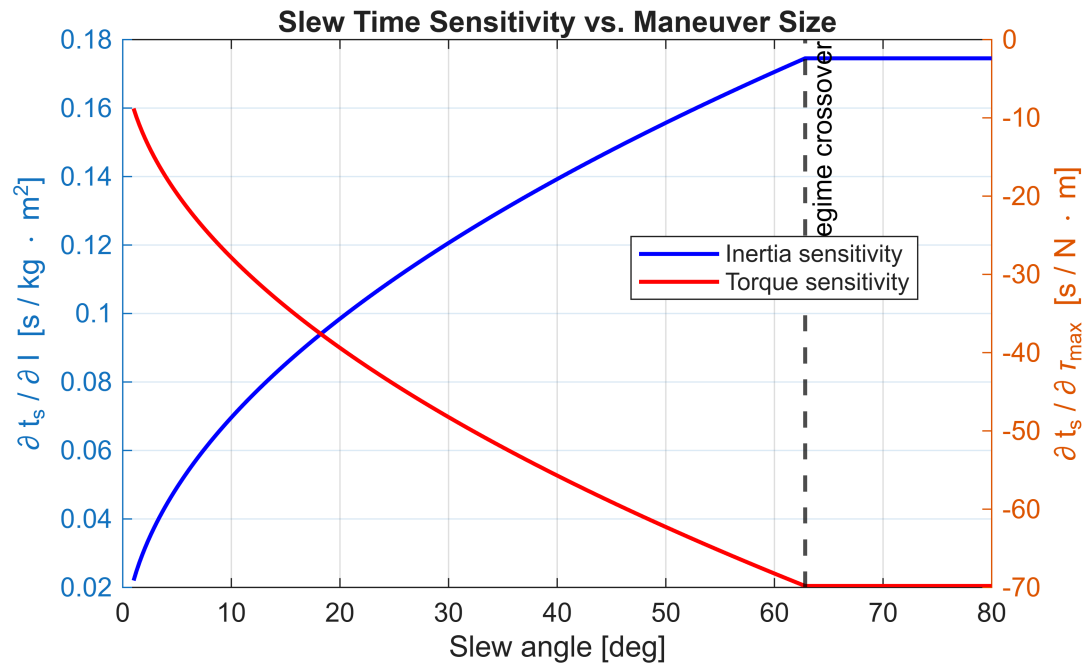
```
=== VPA Verification (32-digit) ===
Analytic ∂t/∂I:      0.12060020909304460989383577121513
Finite-diff (δI=0.001): 0.12059995784365587074682401000842
```

```
fprintf('Agreement to %.0f digits\n', ...
    -log10(double(abs(analytic_sensitivity - fd_sensitivity)/analytic_sensitivity)))
```

Sensitivity Across Slew Angles

The sensitivity structure changes at the regime crossover. Plotting it reveals where inertia growth hurts most.

```
Theta_sweep = linspace(deg2rad(1), deg2rad(80), 300);
dts_dI_sweep = zeros(size(Theta_sweep));
dts_dtau_sweep = zeros(size(Theta_sweep));
for k = 1:length(Theta_sweep)
    [dts_dI_sweep(k), dts_dtau_sweep(k), ~] = slewTimeSensitivity(...
        Theta_sweep(k), I_val, tau_max_val, omega_max_val);
end
clf
yyaxis left
plot(rad2deg(Theta_sweep), dts_dI_sweep, 'b-', 'LineWidth', 1.5)
ylabel('\partial t_s / \partial I [s / kg \cdot m^2]')
yyaxis right
plot(rad2deg(Theta_sweep), dts_dtau_sweep, 'r-', 'LineWidth', 1.5)
ylabel('\partial t_s / \partial \tau_{max} [s / N \cdot m]')
xlabel('Slew angle [deg]')
xline(rad2deg(Theta_cross_val), 'k--', 'Regime crossover', 'LineWidth', 1.5)
title('Slew Time Sensitivity vs. Maneuver Size')
legend('Inertia sensitivity', 'Torque sensitivity', 'Location', 'best')
grid on
```



9. Pointing Budget Decomposition

High-resolution imaging requires sub-arcsecond pointing stability after the slew settles. The total pointing error is the RSS of independent contributors. Each term is a closed-form symbolic expression, enabling exact allocation without requiring Monte Carlo simulation.

Error sources for a typical agile EO satellite:

- Star tracker noise: σ_{ST}
- Reaction wheel jitter (imbalance-induced): σ_{RW}
- Structural flex (appendage modes): σ_{flex}
- Thermal distortion (gradient across bus): σ_{therm}

```
syms sigma_ST sigma_RW sigma_flex sigma_therm positive
```

Total pointing error (RSS):

```
sigma_total = sqrt(sigma_ST^2 + sigma_RW^2 + sigma_flex^2 + sigma_therm^2)
```

$$\sigma_{\text{total}} = \sqrt{\sigma_{\text{ST}}^2 + \sigma_{\text{RW}}^2 + \sigma_{\text{flex}}^2 + \sigma_{\text{therm}}^2}$$

Sensitivity of Pointing Budget to Each Source

The partial derivative $\partial\sigma_{\text{total}}/\partial\sigma_i$ tells the pointing engineer which error source offers the best return on investment for improvement.

```
dsig_dST = simplify(diff(sigma_total, sigma_ST));
dsig_dRW = simplify(diff(sigma_total, sigma_RW));
dsig_dFlex = simplify(diff(sigma_total, sigma_flex));
```

$\partial\sigma_{\text{total}}/\partial\sigma_{\text{ST}}$:

```
dsig_dTherm = simplify(diff(sigma_total, sigma_therm))
```

```
dsig_dTherm =
```

$$\frac{\sigma_{\text{therm}}}{\sqrt{\sigma_{\text{ST}}^2 + \sigma_{\text{RW}}^2 + \sigma_{\text{flex}}^2 + \sigma_{\text{therm}}^2}}$$

Evaluate Pointing Budget

Substitute realistic error allocations for a high-resolution agile EO satellite (0.5 m GSD from 500 km LEO requires total pointing error below ~1 arcsec).

```
sigma_ST_val = 0.3;      % arcsec – high-end star tracker
sigma_RW_val = 0.15;     % arcsec – balanced reaction wheels
sigma_flex_val = 0.2;    % arcsec – stiff CFRP bus
sigma_therm_val = 0.1;   % arcsec – thermal control
sigma_total_val = double(subs(sigma_total, ...
    [sigma_ST, sigma_RW, sigma_flex, sigma_therm], ...
    [sigma_ST_val, sigma_RW_val, sigma_flex_val, sigma_therm_val]));
fprintf(['=== Pointing Budget ===\n' ...
    'Star tracker:      %.2f arcsec\n' ...
    'Wheel jitter:      %.2f arcsec\n' ...
    'Structural flex:   %.2f arcsec\n' ...
```

```

'Thermal distortion: %.2f arcsec\n' ...
'Total (RSS):          %.4f arcsec\n'], ...
sigma_ST_val, ...
sigma_RW_val, ...
sigma_flex_val, ...
sigma_therm_val, ...
sigma_total_val)

```

```

=== Pointing Budget ===
Star tracker:      0.30 arcsec
Wheel jitter:      0.15 arcsec
Structural flex:   0.20 arcsec
Thermal distortion: 0.10 arcsec
Total (RSS):       0.4031 arcsec

```

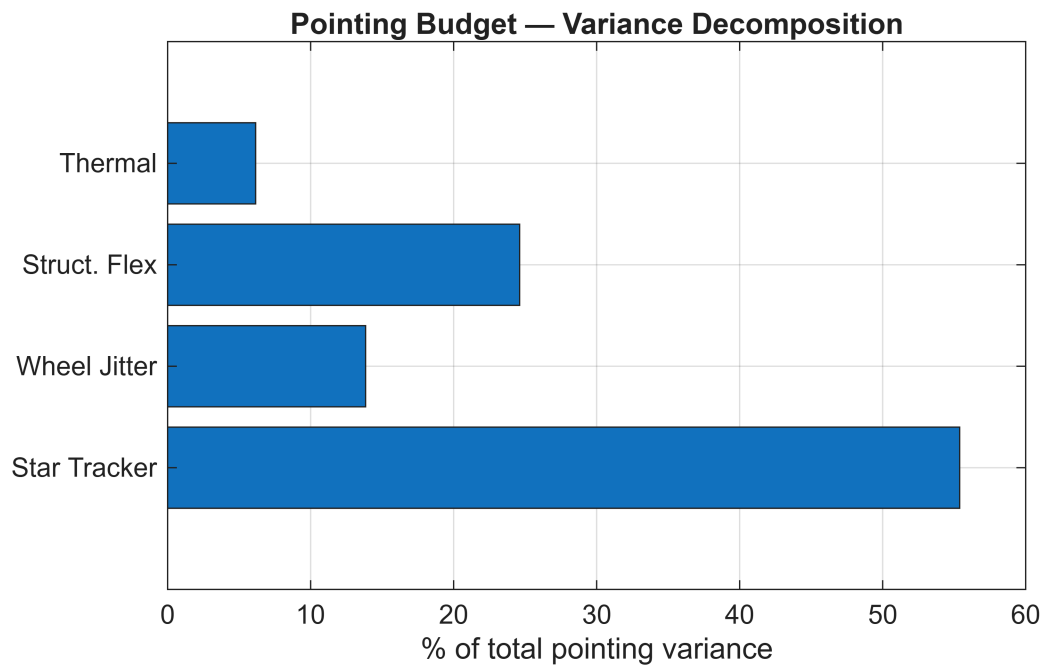
Variance Contribution Breakdown

Identify the dominant error by analyzing each source's fractional contribution to the total variance.

```

variances = [sigma_ST_val^2, sigma_RW_val^2, sigma_flex_val^2, sigma_therm_val^2];
total_var = sum(variances);
pct_contributions = 100 * variances / total_var;
clf
barh(pct_contributions)
set(gca, 'YTick', 1:4, 'YTickLabel', ...
    {'Star Tracker', 'Wheel Jitter', 'Struct. Flex', 'Thermal'})
xlabel('% of total pointing variance')
title('Pointing Budget – Variance Decomposition')
grid on

```



```
fprintf(['\nVariance contributions:\n' ...
        ' Star tracker:    %5.1f%%\n' ...
        ' Wheel jitter:    %5.1f%%\n' ...
        ' Structural flex: %5.1f%%\n' ...
        ' Thermal:         %5.1f%%\n'], ...
        pct_contributions(1), ...
        pct_contributions(2), ...
        pct_contributions(3), ...
        pct_contributions(4))
```

```
Variance contributions:
Star tracker:    55.4%n Wheel jitter:    13.8%n Structural flex: 24.6%n Thermal:    6.2%nn
```

10. Multi-Target Scheduling

The ultimate payoff: given a sequence of imaging targets, the generated slew planner computes all inter-target slew times in a single vectorized call. This feeds directly into scheduling optimizers that maximize daily imaging capacity.

Scenario: 8 imaging targets in one orbital pass, at various off-nadir angles along the ground track.

```
target_angles_deg = [0, 15, -20, 35, -10, 25, -30, 5];
n_targets = length(target_angles_deg);
slew_angles_pass = abs(diff(target_angles_deg));
slew_angles_pass_rad = deg2rad(slew_angles_pass);
t_slews_pass = slewTime(slew_angles_pass_rad, I_val, tau_max_val, omega_max_val);
settle_time = 2.0;
image_time = 8.0;
t_slew_total = sum(t_slews_pass);
t_settle_total = settle_time * (n_targets - 1);
t_image_total = image_time * n_targets;
t_pass_total = t_slew_total + t_settle_total + t_image_total;
fprintf(['=== Multi-Target Pass Timeline ===\n' ...
        'Targets: %d | Orbit pass window: ~600 s\n' ...
        '\nTarget schedule (off-nadir angles):\n' ...
        ' ' ...
        '%+d° → ' ...
        '%+d°\n' ...
        '\nInter-target slews:\n' ...
        ' Slew    Angle    Time\n'], ...
        n_targets, ...
        target_angles_deg(1:end-1), ...
        target_angles_deg(end))
```

```
=== Multi-Target Pass Timeline ===
Targets: 8 | Orbit pass window: ~600 s

Target schedule (off-nadir angles):
+0° → +15°

Inter-target slews:
Slew    Angle    Time
=== Multi-Target Pass Timeline ===
Targets: -20 | Orbit pass window: ~600 s

Target schedule (off-nadir angles):
+35° → -10°

Inter-target slews:
```

```

Slew    Angle    Time
=== Multi-Target Pass Timeline ===
Targets: 25 | Orbit pass window: ~600 s

```

```

Target schedule (off-nadir angles):
-30° → +5°

```

```

Inter-target slews:
Slew    Angle    Time

```

```

for k = 1:length(slew_angles_pass)
    fprintf(' %d→%d    %4.0f°    %5.1f s\n', k, k+1, slew_angles_pass(k), t_slews_pass(k))
end

```

```

1→2    15°    20.5 s
2→3    35°    31.3 s
3→4    55°    39.2 s
4→5    45°    35.4 s
5→6    35°    31.3 s
6→7    55°    39.2 s
7→8    35°    31.3 s

```

```

fprintf(['\n Total slew time:    %5.1f s\n' ...
    ' Total settle time: %5.1f s (%.1f s × %d)\n' ...
    ' Total image time:  %5.1f s (%.1f s × %d)\n' ...
    ' _____\n' ...
    ' Pass total:         %5.1f s\n'], ...
    t_slew_total, ...
    t_settle_total, settle_time, n_targets-1, ...
    t_image_total, image_time, n_targets, ...
    t_pass_total)

```

```

Total slew time:    228.1 s
Total settle time:  14.0 s (2.0 s × 7)
Total image time:   64.0 s (8.0 s × 8)
_____
Pass total:         306.1 s

```

Visualize the Timeline

Build a Gantt-style timeline showing how slew, settle, and imaging segments fill the orbital pass window.

```

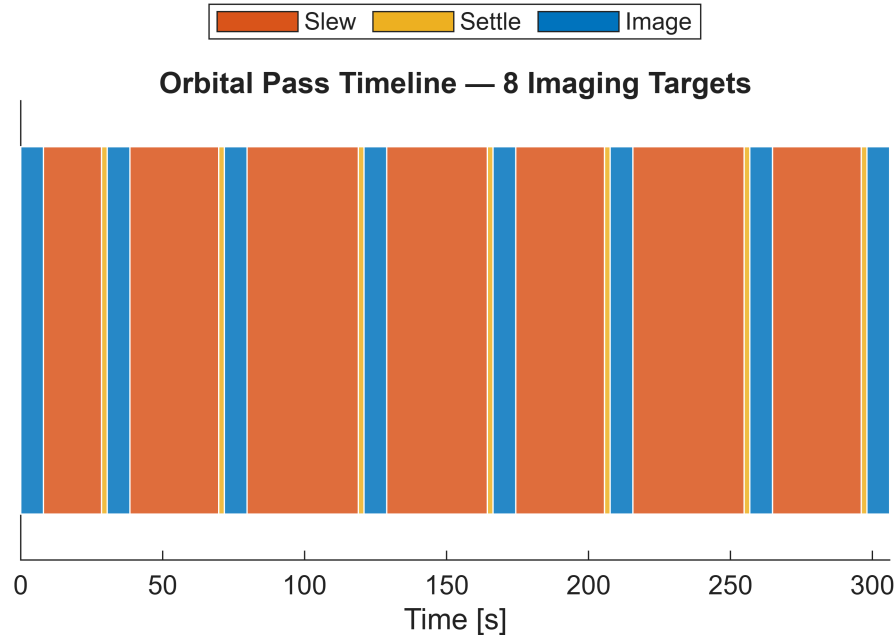
n_segments = 3*n_targets - 2; % image + (slew + settle) for each transition
seg_starts = zeros(1, n_segments);
seg_durations = zeros(1, n_segments);
seg_types = zeros(1, n_segments); % 1=slew, 2=settle, 3=image
t_cumulative = 0;
seg_idx = 0;
for k = 1:n_targets
    if k > 1
        seg_idx = seg_idx + 1;
        seg_starts(seg_idx) = t_cumulative;
        seg_durations(seg_idx) = t_slews_pass(k-1);
        seg_types(seg_idx) = 1;
        t_cumulative = t_cumulative + t_slews_pass(k-1);
        seg_idx = seg_idx + 1;
        seg_starts(seg_idx) = t_cumulative;
        seg_durations(seg_idx) = settle_time;
        seg_types(seg_idx) = 2;
        t_cumulative = t_cumulative + settle_time;
    end
    seg_idx = seg_idx + 1;
    seg_starts(seg_idx) = t_cumulative;
    seg_durations(seg_idx) = image_time;
    seg_types(seg_idx) = 3;
    t_cumulative = t_cumulative + image_time;
end
colors_tl = [0.85 0.33 0.10; 0.93 0.69 0.13; 0.00 0.45 0.74];
clf
hold on
for s = 1:seg_idx
    patch([seg_starts(s), seg_starts(s)+seg_durations(s), ...
           seg_starts(s)+seg_durations(s), seg_starts(s)], ...
          [0.1, 0.1, 0.9, 0.9], colors_tl(seg_types(s,:),:), ...
          'FaceAlpha', 0.85, 'EdgeColor', 'w', 'LineWidth', 0.5, ...
          'HandleVisibility', 'off')
end
patch(nan, nan, colors_tl(1,:), 'DisplayName', 'Slew')
patch(nan, nan, colors_tl(2,:), 'DisplayName', 'Settle')

```

```

patch(nan, nan, colors_t1(3,:), 'DisplayName', 'Image')
hold off
xlabel('Time [s]')
title('Orbital Pass Timeline — 8 Imaging Targets')
legend('Location', 'northoutside', 'Orientation', 'horizontal')
set(gca, 'YTick', [], 'YLim', [0 1])
xlim([0, t_pass_total])

```



Impact of Degradation on Mission Capacity

Re-evaluate the pass timeline under end-of-life conditions (inertia +10%, torque -15%) to quantify how many targets the spacecraft can still service within the pass window.

```

I_eol = I_val * 1.10;
tau_eol = tau_max_val * 0.85;
t_slews_eol = slewTime(slew_angles_pass_rad, I_eol, tau_eol, omega_max_val);
t_pass_eol = sum(t_slews_eol) + t_settle_total + t_image_total;
fprintf(['\n=== End-of-Life Timeline Impact ===\n' ...

```

```
'BOL pass total: %.1f s\n'], ...
t_pass_total)
```

```
=== End-of-Life Timeline Impact ===
BOL pass total: 306.1 s
```

```
fprintf('EOL pass total: %.1f s (+%.1f s, +%.1f%%)\n', ...
t_pass_eol, t_pass_eol - t_pass_total, ...
100*(t_pass_eol - t_pass_total)/t_pass_total)
```

```
EOL pass total: 337.5 s (+31.4 s, +10.3%)
```

```
pass_window = 600;
if t_pass_eol > pass_window
    t_cumulative_eol = image_time;
    max_targets_eol = 1;
    for k = 1:length(slew_angles_pass)
        t_cumulative_eol = t_cumulative_eol + t_slews_eol(k) + settle_time + image_time;
        if t_cumulative_eol > pass_window
            break;
        end
        max_targets_eol = max_targets_eol + 1;
    end
    fprintf(['Pass window: %.0f s\n' ...
'EOL capacity: %d of %d targets (%.0f%% utilization)\n'], ...
pass_window, max_targets_eol, n_targets, 100*max_targets_eol/n_targets)
else
    fprintf('EOL: all %d targets still fit within %.0f s window\n', n_targets, pass_window)
end
```

```
EOL: all 8 targets still fit within 600 s window
```

Parametric Exploration of Slew Time Trade Space

The symbolic slew planner enables instant parametric sweeps. Here we show slew time vs. angle for multiple torque levels, capturing the full actuator sizing trade.

```
Theta_sweep_fine = linspace(deg2rad(0.5), deg2rad(60), 500);
tau_levels = [0.15, 0.20, 0.30, 0.50];
```

```

clf
hold on
for k = 1:length(tau_levels)
    t_sweep_k = slewTime(Theta_sweep_fine, I_val, tau_levels(k), omega_max_val);
    plot(rad2deg(Theta_sweep_fine), t_sweep_k, 'LineWidth', 2, ...
        'DisplayName', sprintf('%.2f N\dot{m}', tau_levels(k)))
end
xline(rad2deg(Theta_cross_val), 'k--', 'HandleVisibility', 'off')
hold off
xlabel('Slew angle [deg]')
ylabel('Slew time [s]')
title('Slew Time Trade Space — Actuator Sizing ( $\tau_{\max}$ )')
legend('Location', 'northwest')
grid on

```

